Internship Recos

High-Level Design Document

Automated genuine-internship aggregation platform · Apify scraping · Normalisation · Validation · Prepared 13 October 2026

1 Purpose & Scope

Internship Recos is a website / app that automatically collects internship listings from external applications using Apify actors on a schedule. The backend normalises and validates the data, filters out fake listings, and stores it in a database; students search the validated listings through the front end. This document covers the system architecture, overall workflow, database schema and supporting design choices at a high level.

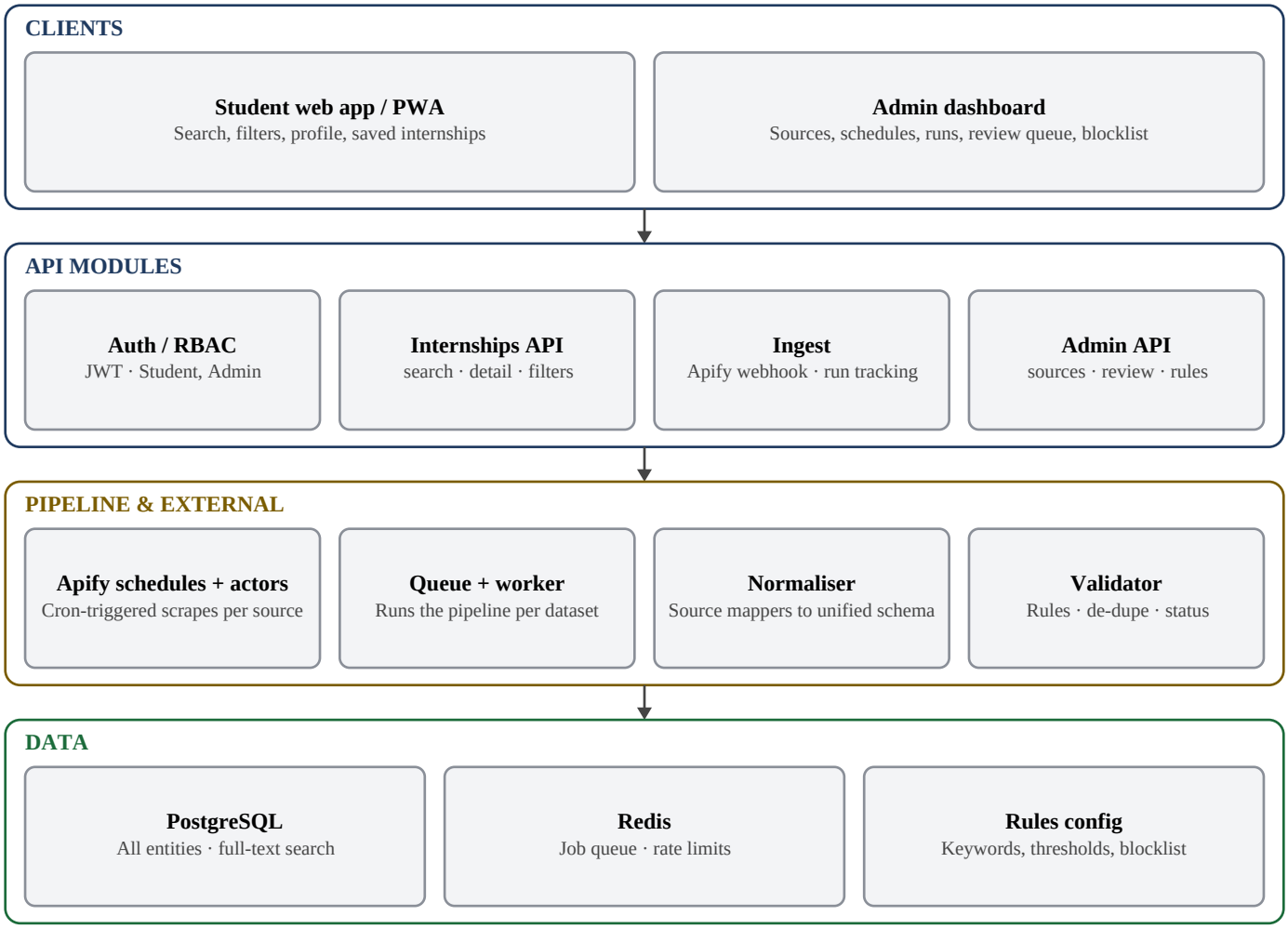
In scope	Out of scope
Scheduled scraping of external sources through Apify actors	Applying inside the platform (we link to the original apply URL)
Raw storage, normalisation to one unified schema	Running our own scraping infrastructure
Validation, fake-listing filtering, de-duplication, expiry	Training custom ML models
Student accounts, profile, search, filters, saved internships	Payments, in-app chat, resume tools, employer portal
Admin dashboard: sources, schedules, run history, review queue, blocklist	Scraping behind logins or against a source's terms

Assumption: the specific Apify actors per source are chosen from the Apify Store (or built as custom actors) after reviewing each source's terms.

At a glance

Layer	What it does
Front end	Students search and filter validated internships, keep a profile and a saved list; admins manage sources and review flagged listings
Apify	Runs the scraping actors on a cron schedule and returns raw listings
Backend	Receives scrape results, normalises and validates them, de-duplicates, publishes, and serves the search API
Database	PostgreSQL stores raw payloads, normalised internships, validation results, users and saved lists

2 System Architecture

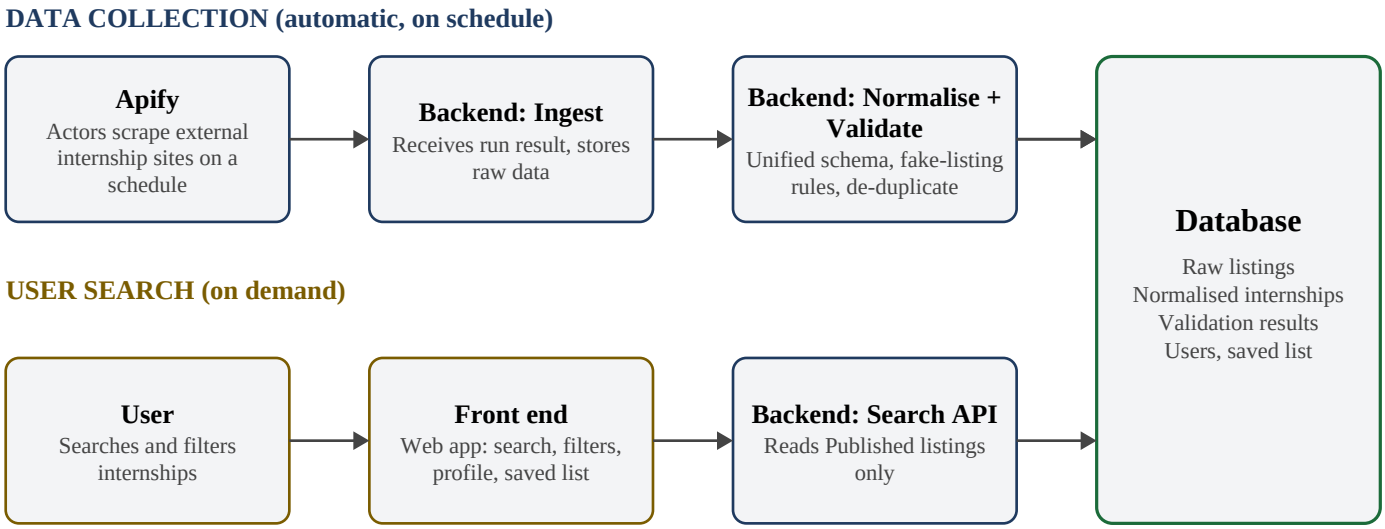


Request path: Client → HTTPS → auth → Internships API → PostgreSQL → results. **Ingestion path:** Apify schedule → actor run → webhook → queue → worker (stage raw → normalise → validate → de-dupe → publish). Students never trigger scrapes; search always reads from the database.

Component	Responsibility
Student web app / PWA	Search with filters, internship detail with source link, profile, saved list
Admin dashboard	Add or pause sources, edit schedules, view runs and errors, review queue, blocklist
Ingest + worker	Verify Apify webhooks (idempotent on run id), fetch datasets, run the pipeline in batches with retries
Normaliser / Validator	Per-source mappers to one schema; hard and soft rules from versioned config; outcomes recorded
Internships API	Search, filters and detail for Published listings; profile and saved list
PostgreSQL / Redis	System of record with search indexes; job queue and rate limits
Apify platform	Hosts actors, schedules, proxies and datasets; our system only consumes actor output

3 Overall Workflow

The front end lets users search, Apify does the web scraping, the backend normalises and validates the scraped data, and everything is stored in the database.



Data collection pipeline in detail

#	Stage	What happens	Stored in
1	Schedule	Each source has its own cron (e.g. every 6 h for busy boards, daily for others); admin can also trigger a run.	scrape_runs
2	Scrape	Apify actor runs with configured input and an item cap, and writes to an Apify dataset.	Apify dataset
3	Webhook	Apify calls the Ingest endpoint with a shared secret; the handler is idempotent on run id and enqueues a job.	scrape_runs
4	Stage raw	Worker saves every item unchanged as JSON with a content hash; unchanged items are skipped.	raw_listings
5	Normalise	Source-specific mapper converts fields to the unified schema.	internships (draft)
6	Validate	Hard rules reject, soft rules raise flags; each outcome is saved with a reason code.	validation_results
7	De-duplicate	Fingerprint plus fuzzy match; same internship from several sources is merged, all links kept.	internship_sources
8	Publish	No hard failure and no flags: Published. Flagged: Needs review for an admin.	internships.status
9	Maintain	Daily sweep expires listings past deadline or unseen in two runs; run counters are closed.	internships, scrape_runs

Admins review flagged listings and blocklist bad companies from the dashboard; those decisions feed back into the rules for every later run.

4 Normalisation & Validation

4.1 Normalisation to one unified schema

Field	Rule
title, company	Clean text; role category from a keyword map; canonical company name (strip Pvt / Ltd) and website domain
location, work_mode	City from an alias map; mode set to onsite / hybrid / remote
stipend, duration	Parsed to min, max, currency (INR default) and months (weeks ÷ 4.3)
dates	posted_at, start_date, apply_by to UTC dates; relative text such as "3 days ago" resolved
skills, description	Skills matched against a curated dictionary; HTML stripped; raw payload kept untouched
apply_url, fingerprint	Tracking parameters removed; hash of company + title + city + start month for de-duplication

4.2 Validation rules

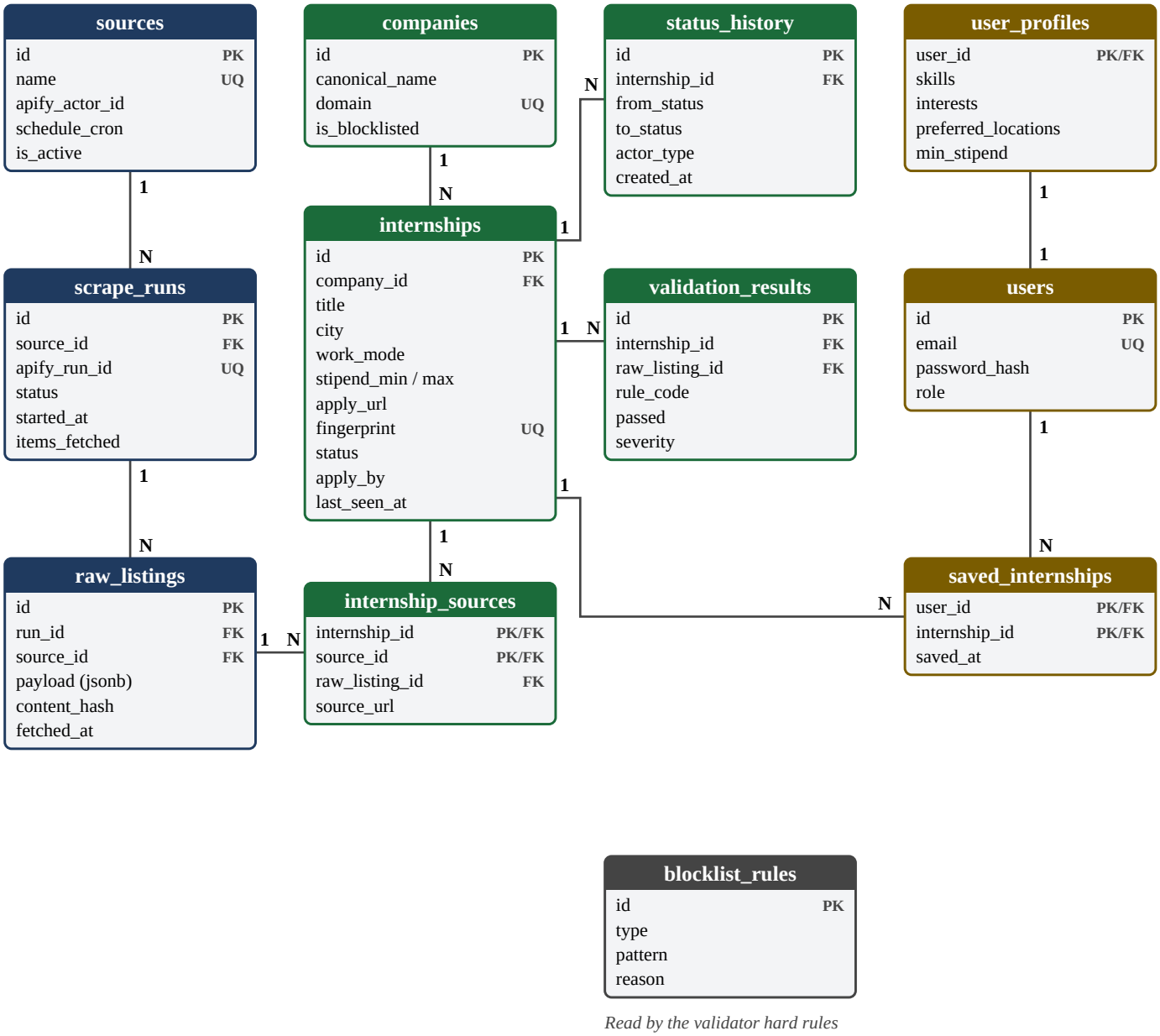
Type	Rule	Outcome on failure
Hard	Required fields present (title, company, apply_url, location or mode); apply_url well-formed	Rejected
Hard	No fee-demand wording (registration fee, deposit, pay to join)	Rejected
Hard	Company or domain not in blocklist	Rejected
Hard	apply_by not already in the past	Expired
Soft	Apply URL reachable; not a link shortener	Flag
Soft	Company website resolves; apply or contact domain matches it	Flag
Soft	Stipend plausible for role and city; description specific, not boilerplate	Flag
Soft	Contact only via WhatsApp / Telegram / free-mail, no official apply route	Flag

4.3 Outcome and failure handling

Status / situation	Behaviour
Published	No hard failure and no flags; visible to students
Needs review / Rejected	Flagged listings wait in the admin queue; rejected ones are kept for audit, never deleted
Expired	Deadline passed or unseen in two consecutive runs; hidden from search
Run failed or partial	Retried with backoff (max 3); partial runs never trigger expiry; source shown as unhealthy
Source layout change	Mapper error list; admin alerted if over 20% of a run fails; raw data can be re-normalised later

5 Database Schema

PostgreSQL, third normal form. Raw payloads and status history are append-only.



Navy: ingestion tables. Green: listing and validation tables. Brown: user tables. ***PK*** *primary key,* ***FK*** *foreign key,* ***UQ*** *unique;* ***1*** *and* ***N*** *mark the ends of each relationship. Key columns only.*

6 Tables, API & Technology

6.1 Tables at a glance

Table	Purpose
sources, scrape_runs	One row per source (actor, input, cron); one per Apify run with status and counters
raw_listings	Every scraped item as received (jsonb + content hash); append-only
companies, internships	Canonical companies; de-duplicated listings with status, fingerprint and search vector
internship_sources	Which sources list each internship, with source URL and last-seen time
validation_results, status_history	Every rule outcome with reason code; audit log of status changes and who made them
blocklist_rules	Blocked domains, companies and keywords used by the hard rules
users, user_profiles, saved_internships	Accounts and roles; profile used to pre-fill filters; bookmarks

6.2 API

Router	Main endpoints	Auth
/auth	POST register, login	Public
/internships	GET search with filters; GET detail	Student
/profile, /saved	GET / PUT profile; POST / DELETE / GET saved	Student
/ingest	POST apify-webhook (idempotent)	Shared secret
/admin	Sources and manual runs, run history, review queue, publish / reject / blocklist, blocklist rules	Admin

6.3 Technology & non-functional notes

Area	Choice / approach
Stack (proposed)	Apify actors; Python FastAPI backend; Celery + Redis worker; PostgreSQL; React + Vite + Tailwind PWA
Reliability	Idempotent webhook, retries with backoff, error list for failed items
Security	JWT roles on every endpoint, webhook secret, rate limits, secrets in environment variables
Compliance & cost	Public data only, per-source terms review, link back to source; item caps and per-source schedules
Observability	Run history, error-rate and stale-source alerts on the admin dashboard